ARCHITECTURE DOCUMENT

Bloom

PARFUMS

Technical Execution Plan

VERSION 1.0 — IMPLEMENTATION READY

Prepared by: Principal Full-Stack Architect AI

Date: February 25, 2026

Source: FULLSTACK_ARCHITECTURE_REPORT.md v3.0

Status: FINAL — No revisions pending

NEXT.JS APP ROUTER | LARAVEL 11 + SANCTUM | MYSQL 8.0 | 13 TABLES | COOKIE WISHLIST | ADMIN-ONLY AUTH

Technical Execution Plan — Bloom Parfums

Document Type: Architecture Execution Plan
Version: 1.0
Date: February 25, 2026
Source: Derived from `FULLSTACK_ARCHITECTURE_REPORT.md` v3.0
Scope: Next.js App Router + Laravel 11 API + MySQL 8.0
Status: FINAL — Implementation-Ready

Table of Contents

1. Applied Architecture Decisions
2. Final Database Schema
3. Backend Implementation Strategy
4. Admin Authentication Model
5. Wishlist Cookie System
6. API Contract
7. Frontend Integration Rules
8. Security & Risk Mitigation
9. Final Technical Notes

1. Applied Architecture Decisions

1.1 Source of Truth Extraction

The following is extracted from the v3.0 report with final decisions locked. No deviations.

Entities confirmed for persistence (12 tables + migrations):

Entity	Justification
`admins`	Single authenticatable actor. Credentials must be persisted. No in-memory alternative.
`brands`	Catalog taxonomy. Required for filter sidebar, product FK, URL structure.
`categories`	Catalog taxonomy. Self-referential hierarchy. Required for filter and product FK.
`products`	Core sellable entity. All catalog operations originate here.
`product_images`	1-N gallery per product. Current flat `image_url` is structurally insufficient.
`product_sizes`	Per-size stock + pricing. Required by checkout and order line items.
`shipping_methods`	Admin-configurable delivery options. Displayed at checkout. FK on orders.
`coupons`	Discount codes with usage tracking, expiry, and min-order constraints.
`orders`	Guest order record. Guest address absorbed inline — no separate address table.

Entity	Justification
`order_items`	Line items with price-at-purchase snapshot. Required for order history and tracking.
`order_status_histories`	Append-only status log. Powers `/order-status` timeline. Required for tracking.
`reviews`	Customer-submitted ratings. Moderation gate (`is_approved`). Product rating aggregations.
`review_images`	Photos attached to reviews. Size-validated, CDN-served. FK to reviews with CASCADE.

Entities explicitly NOT persisted:

Entity	Decision	Rationale
`wishlists`	**Cookie only**	No user accounts. Persistent wishlist requires a user identity. 30-day TTL satisfies UX requirement. Backend role is validation only — product IDs resolved on demand.
`users` / `customers`	**Dropped entirely**	The system has no public auth. The current `users` migration and `User` model are removed. No customer registration. No customer session.
`sessions`	**Dropped**	Sanctum token auth does not require a sessions table for API-only operation.
`cart`	**Zustand (in-memory)**	Cart is ephemeral. It is populated during the browsing session and consumed at checkout. No backend persistence. The order creation endpoint receives the cart as input — it never stores a draft cart.
`password_reset_tokens`	**Dropped**	Admin password reset is handled via a seeder or Artisan command. No public-facing reset flow.

1.2 Backend Feature Set (Final)

Derived from functional requirements in Section 4 of the report. No invented features.

Public (unauthenticated) backend responsibilities:

- Serve product catalog (list + detail + wishlist validation)
- Serve brands, categories, shipping methods
- Validate coupon codes against subtotal
- Create guest orders (with full server-side price resolution)
- Track orders by order_number + phone match
- Accept review submissions (unapproved by default)
- Serve approved reviews per product

Admin (Sanctum-authenticated) backend responsibilities:

- Admin login / logout
- Full product CRUD (with image and size management)
- Brand CRUD, Category CRUD, Coupon CRUD
- Shipping method management
- Order list + status update (appends to history)
- Review moderation (approve / reject)

Backend never does:

- Store cart state
- Store wishlist state
- Issue public JWT tokens

- Register customers
- Accept user-side pricing data

1.3 Superseded Decisions from Current Codebase

Superseded Element	Replacement	Action
`tymon/jwt-auth ^2.1`	`laravel/sanctum ^4.0`	`composer remove tymon/jwt-auth` → `composer require laravel/sanctum`
`User` model (customer `HasMany` orders, role enum)	`Admin` model (Sanctum `HasApiTokens`)	Drop users table, create admins
`lib/auth.ts` `setAuthToken()` (js-cookie write)	Admin token in HttpOnly cookie (server-side Set-Cookie)	Remove client-side token write entirely
`services/api.ts` Bearer interceptor from js-cookie	`withCredentials: true` (Sanctum cookie sent automatically)	Remove `Authorization: Bearer` header logic
Inline `authorizeAdmin()` in ProductController	`EnsureAdmin` middleware on route group	Move auth check out of business logic
`Product.category` (free text string)	`category_id` FK	Data migration + schema change
`/register` route + `POST /auth/register`	**Removed**	No public registration path

2. Final Database Schema

2.1 Entity Inclusion/Exclusion Rationale

Wishlist is NOT in the database — definitive justification:

The wishlist is a client preference list, not a business transaction. It has no financial consequence until an order is placed. Storing it requires a user identity, which this system explicitly has none of. The 30-day cookie TTL provides adequate persistence for the shopping use case. Backend involvement is limited to a product validation endpoint — it resolves IDs to current product state but writes nothing.

Admins ARE in the database — definitive justification:

Authentication requires a persistent credential store. The admin email/password hash must survive server restarts, deployments, and process kills. An in-memory credential is not a production pattern. The `admins` table has exactly 1 row in production.

2.2 Complete DDL

```
-- ============================================================
-- BLOOM PARFUMS — Technical Execution Plan
-- Final Production Schema v1.0
-- Engine: MySQL 8.0+ (InnoDB), Charset: utf8mb4_unicode_ci
-- ============================================================

SET FOREIGN_KEY_CHECKS = 0;

-- ── admins ──────────────────────────────────────────────────
CREATE TABLE admins (
  id         BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  email      VARCHAR(191)     NOT NULL,
  password   VARCHAR(255)     NOT NULL,
  created_at TIMESTAMP        NULL DEFAULT NULL,
  updated_at TIMESTAMP        NULL DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_admins_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── brands ──────────────────────────────────────────────────
CREATE TABLE brands (
  id         BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  name       VARCHAR(100)     NOT NULL,
  slug       VARCHAR(120)     NOT NULL,
  logo_url   VARCHAR(500)     NULL,
  created_at TIMESTAMP        NULL DEFAULT NULL,
  updated_at TIMESTAMP        NULL DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_brands_slug (slug)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── categories ──────────────────────────────────────────────
CREATE TABLE categories (
  id         BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  parent_id  BIGINT UNSIGNED  NULL,
  name       VARCHAR(100)     NOT NULL,
  slug       VARCHAR(120)     NOT NULL,
  sort_order TINYINT UNSIGNED NOT NULL DEFAULT 0,
  created_at TIMESTAMP        NULL DEFAULT NULL,
  updated_at TIMESTAMP        NULL DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_categories_slug (slug),
  KEY idx_categories_parent (parent_id),
  CONSTRAINT fk_categories_parent
    FOREIGN KEY (parent_id) REFERENCES categories (id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── products ────────────────────────────────────────────────
CREATE TABLE products (
  id             BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  brand_id       BIGINT UNSIGNED  NOT NULL,
  category_id    BIGINT UNSIGNED  NOT NULL,
  name           VARCHAR(255)     NOT NULL,
  slug           VARCHAR(300)     NOT NULL,
  subtitle       VARCHAR(255)     NULL,
  description    TEXT             NOT NULL,
  ingredients    TEXT             NULL,
  gender         ENUM('man','woman','child','unisex') NOT NULL DEFAULT 'unisex',
  price          DECIMAL(10,2)    NOT NULL,
  original_price DECIMAL(10,2)    NULL,
  stock          INT UNSIGNED     NOT NULL DEFAULT 0,
  is_active      TINYINT(1)       NOT NULL DEFAULT 1,
  is_featured    TINYINT(1)       NOT NULL DEFAULT 0,
  created_at     TIMESTAMP        NULL DEFAULT NULL,
  updated_at     TIMESTAMP        NULL DEFAULT NULL,
  deleted_at     TIMESTAMP        NULL DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_products_slug (slug),
  KEY idx_products_brand    (brand_id),
  KEY idx_products_category (category_id),
```

```
  KEY idx_products_listing  (is_active, is_featured, deleted_at),
  KEY idx_products_gender   (gender),
  KEY idx_products_price    (price),
  CONSTRAINT fk_products_brand
    FOREIGN KEY (brand_id)    REFERENCES brands     (id) ON DELETE RESTRICT,
  CONSTRAINT fk_products_category
    FOREIGN KEY (category_id) REFERENCES categories (id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── product_images ──────────────────────────────────
CREATE TABLE product_images (
  id         BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  product_id BIGINT UNSIGNED  NOT NULL,
  url        VARCHAR(500)     NOT NULL,
  alt        VARCHAR(255)     NULL,
  sort_order TINYINT UNSIGNED NOT NULL DEFAULT 0,
  is_primary TINYINT(1)       NOT NULL DEFAULT 0,
  created_at TIMESTAMP        NULL DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_product_images_product (product_id),
  KEY idx_product_images_primary (product_id, is_primary),
  CONSTRAINT fk_product_images_product
    FOREIGN KEY (product_id) REFERENCES products (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── product_sizes ───────────────────────────────────
CREATE TABLE product_sizes (
  id             BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  product_id     BIGINT UNSIGNED  NOT NULL,
  label          VARCHAR(50)      NOT NULL,
  price_modifier DECIMAL(8,2)     NOT NULL DEFAULT 0.00,
  stock          INT UNSIGNED     NOT NULL DEFAULT 0,
  created_at     TIMESTAMP        NULL DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_product_sizes_product (product_id),
  UNIQUE KEY uq_product_sizes_label (product_id, label),
  CONSTRAINT fk_product_sizes_product
    FOREIGN KEY (product_id) REFERENCES products (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── shipping_methods ────────────────────────────────
CREATE TABLE shipping_methods (
  id          BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  name        VARCHAR(100)     NOT NULL,
  description VARCHAR(255)     NULL,
  price       DECIMAL(8,2)     NOT NULL,
  free_over   DECIMAL(10,2)    NULL,
  is_active   TINYINT(1)       NOT NULL DEFAULT 1,
  sort_order  TINYINT UNSIGNED NOT NULL DEFAULT 0,
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── coupons ─────────────────────────────────────────
CREATE TABLE coupons (
  id               BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  code             VARCHAR(50)      NOT NULL,
  type             ENUM('fixed','percent') NOT NULL,
  value            DECIMAL(8,2)     NOT NULL,
  min_order_amount DECIMAL(10,2)    NULL,
  usage_limit      INT UNSIGNED     NULL,
  used_count       INT UNSIGNED     NOT NULL DEFAULT 0,
  expires_at       TIMESTAMP        NULL DEFAULT NULL,
  is_active        TINYINT(1)       NOT NULL DEFAULT 1,
  created_at       TIMESTAMP        NULL DEFAULT NULL,
  updated_at       TIMESTAMP        NULL DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_coupons_code (code),
  KEY idx_coupons_active (is_active, expires_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── orders ──────────────────────────────────────────
CREATE TABLE orders (
  id                BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
```

```
  order_number       VARCHAR(20)      NOT NULL,
  shipping_method_id BIGINT UNSIGNED  NOT NULL,
  coupon_id          BIGINT UNSIGNED  NULL,
  first_name         VARCHAR(100)     NOT NULL,
  last_name          VARCHAR(100)     NOT NULL,
  phone              VARCHAR(20)      NOT NULL,
  city               VARCHAR(100)     NOT NULL,
  quartier           VARCHAR(100)     NULL,
  zip_code           VARCHAR(20)      NULL,
  address_line       VARCHAR(500)     NOT NULL,
  status             ENUM('pending','confirmed','dispatched','shipped','delivered','cancelled')
                                      NOT NULL DEFAULT 'pending',
  subtotal           DECIMAL(10,2)    NOT NULL,
  shipping_cost      DECIMAL(10,2)    NOT NULL DEFAULT 0.00,
  discount_amount    DECIMAL(10,2)    NOT NULL DEFAULT 0.00,
  total              DECIMAL(10,2)    NOT NULL,
  notes              TEXT             NULL,
  created_at         TIMESTAMP        NULL DEFAULT NULL,
  updated_at         TIMESTAMP        NULL DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_orders_number (order_number),
  KEY idx_orders_phone   (phone),
  KEY idx_orders_status  (status),
  KEY idx_orders_created (created_at),
  CONSTRAINT fk_orders_shipping
    FOREIGN KEY (shipping_method_id) REFERENCES shipping_methods (id) ON DELETE RESTRICT,
  CONSTRAINT fk_orders_coupon
    FOREIGN KEY (coupon_id)          REFERENCES coupons           (id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── order_items ──────────────────────────────────────────
CREATE TABLE order_items (
  id         BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  order_id   BIGINT UNSIGNED  NOT NULL,
  product_id BIGINT UNSIGNED  NOT NULL,
  size_label VARCHAR(50)      NULL,
  quantity   INT UNSIGNED     NOT NULL,
  unit_price DECIMAL(10,2)    NOT NULL,
  created_at TIMESTAMP        NULL DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_order_items_order   (order_id),
  KEY idx_order_items_product (product_id),
  CONSTRAINT fk_order_items_order
    FOREIGN KEY (order_id)   REFERENCES orders   (id) ON DELETE CASCADE,
  CONSTRAINT fk_order_items_product
    FOREIGN KEY (product_id) REFERENCES products (id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── order_status_histories ───────────────────────────────
CREATE TABLE order_status_histories (
  id         BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  order_id   BIGINT UNSIGNED  NOT NULL,
  status     VARCHAR(50)      NOT NULL,
  label      VARCHAR(255)     NOT NULL,
  location   VARCHAR(255)     NULL,
  created_at TIMESTAMP        NULL DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_osh_order (order_id),
  CONSTRAINT fk_osh_order
    FOREIGN KEY (order_id) REFERENCES orders (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── reviews ──────────────────────────────────────────────
CREATE TABLE reviews (
  id            BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  product_id    BIGINT UNSIGNED  NOT NULL,
  order_number  VARCHAR(20)      NULL,
  reviewer_name VARCHAR(100)     NOT NULL,
  rating        TINYINT UNSIGNED NOT NULL,
  body          TEXT             NULL,
  is_approved   TINYINT(1)       NOT NULL DEFAULT 0,
  approved_at   TIMESTAMP        NULL DEFAULT NULL,
  created_at    TIMESTAMP        NULL DEFAULT NULL,
```

```
  updated_at    TIMESTAMP        NULL DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_reviews_product_approved (product_id, is_approved),
  KEY idx_reviews_order_number     (order_number),
  CONSTRAINT fk_reviews_product
    FOREIGN KEY (product_id) REFERENCES products (id) ON DELETE CASCADE,
  CONSTRAINT chk_reviews_rating CHECK (rating BETWEEN 1 AND 5)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── review_images ──────────────────────────────────────
CREATE TABLE review_images (
  id         BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  review_id  BIGINT UNSIGNED NOT NULL,
  url        VARCHAR(500)    NOT NULL,
  created_at TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_review_images_review (review_id),
  CONSTRAINT fk_review_images_review
    FOREIGN KEY (review_id) REFERENCES reviews (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

SET FOREIGN_KEY_CHECKS = 1;
```

2.3 Migration Execution Order

Respect FK dependency chain. Create in this sequence:

```
1. admins
2. brands
3. categories            (self-referential FK — acceptable, SET NULL on delete)
4. products              (FK → brands, categories)
5. product_images        (FK → products)
6. product_sizes         (FK → products)
7. shipping_methods
8. coupons
9. orders                (FK → shipping_methods, coupons)
10. order_items          (FK → orders, products)
11. order_status_histories (FK → orders)
12. reviews              (FK → products)
13. review_images        (FK → reviews)
```

2.4 Soft Delete Policy

Soft deletes are applied only where product deletion has downstream referential consequences:

Table	Soft Delete	Reason
`products`	YES (`deleted_at`)	Active orders and reviews reference deleted products. Hard delete breaks historical data.
`orders`	NO	Orders are never deleted. Status `cancelled` handles end-of-life.
`reviews`	NO	Admin moderation uses `is_approved` flag, not deletion.
`admins`	NO	Single-row table. No deletion expected. Deactivation is an admin panel concern.
All others	NO	No downstream reference risk.

3. Backend Implementation Strategy

3.1 Layer Responsibilities

Each layer has a single, non-negotiable responsibility. Violations are rejected at code review.

Controllers receive the HTTP request, delegate to a service or query, return a resource response. No business logic. No direct DB queries.

Services contain all multi-step business logic. `OrderService`, `CouponService`, `ReviewService`, `ProductSearchService`. All DB transactions live here.

Form Requests validate input before the controller method body executes. Controllers never call `$request->validate()` inline.

Resources shape the JSON output. Controllers never build arrays manually. Resources prevent column leakage.

Policies define authorization rules scoped to admin actions. Used for per-resource checks within admin routes.

Middleware handles cross-cutting concerns: ForceJsonResponse, EnsureAdmin, SecurityHeaders, throttle.

3.2 Target Directory Structure

```
app/
├── Http/
│   ├── Controllers/
│   │   └── Api/V1/
│   │       ├── Admin/
│   │       │   ├── AdminAuthController.php
│   │       │   ├── ProductController.php
│   │       │   ├── BrandController.php
│   │       │   ├── CategoryController.php
│   │       │   ├── CouponController.php
│   │       │   ├── OrderController.php
│   │       │   └── ReviewController.php
│   │       ├── ProductController.php
│   │       ├── BrandController.php
│   │       ├── CategoryController.php
│   │       ├── ShippingMethodController.php
│   │       ├── OrderController.php
│   │       ├── ReviewController.php
│   │       └── CouponController.php
│   ├── Middleware/
│   │   ├── ForceJsonResponse.php      ← exists, keep
│   │   ├── EnsureAdmin.php            ← new
│   │   └── SecurityHeaders.php        ← new
│   ├── Requests/
│   │   ├── Admin/
│   │   │   ├── LoginRequest.php
│   │   │   ├── StoreProductRequest.php
│   │   │   ├── UpdateProductRequest.php
│   │   │   ├── StoreBrandRequest.php
│   │   │   ├── StoreCategoryRequest.php
│   │   │   ├── StoreCouponRequest.php
│   │   │   └── UpdateOrderStatusRequest.php
│   │   ├── StoreOrderRequest.php
│   │   ├── StoreReviewRequest.php
│   │   └── ValidateCouponRequest.php
│   └── Resources/
│       ├── ProductResource.php          ← lean (listing)
│       ├── ProductDetailResource.php    ← full (detail page)
│       ├── BrandResource.php
│       ├── CategoryResource.php
│       ├── ShippingMethodResource.php
│       ├── OrderResource.php
│       ├── OrderTrackResource.php       ← redacted (public tracking)
│       ├── ReviewResource.php
│       └── CouponValidationResource.php
├── Models/
│   ├── Admin.php
│   ├── Brand.php
│   ├── Category.php
│   ├── Product.php
│   ├── ProductImage.php
│   ├── ProductSize.php
│   ├── ShippingMethod.php
│   ├── Coupon.php
│   ├── Order.php
│   ├── OrderItem.php
│   ├── OrderStatusHistory.php
│   ├── Review.php
│   └── ReviewImage.php
├── Services/
│   ├── OrderService.php
│   ├── CouponService.php
│   ├── ReviewService.php
│   └── ProductSearchService.php
└── Jobs/
    ├── SendOrderConfirmationSms.php
    └── ProcessReviewImage.php
```

3.3 Model Contracts

```
// Admin.php
class Admin extends Authenticatable {
    use HasApiTokens, HasFactory;
    protected $fillable = ['email', 'password'];
    protected $hidden   = ['password', 'remember_token'];
    protected $casts    = ['password' => 'hashed'];
}

// Product.php
class Product extends Model {
    use SoftDeletes;

    protected $fillable = [
        'brand_id','category_id','name','slug','subtitle',
        'description','ingredients','gender',
        'price','original_price','stock','is_active','is_featured',
    ];

    protected $casts = [
        'price'          => 'decimal:2',
        'original_price' => 'decimal:2',
        'is_active'      => 'boolean',
        'is_featured'    => 'boolean',
    ];

    protected static function booted(): void {
        static::creating(function (Product $p) {
            $p->slug ??= Str::slug($p->name) . '-' . Str::lower(Str::random(6));
        });
    }

    public function brand():    BelongsTo { return $this->belongsTo(Brand::class); }
    public function category(): BelongsTo { return $this->belongsTo(Category::class); }
    public function images():   HasMany   { return $this->hasMany(ProductImage::class)->orderBy('sort_order'); }
    public function sizes():    HasMany   { return $this->hasMany(ProductSize::class); }
    public function reviews():  HasMany   { return $this->hasMany(Review::class)->where('is_approved', true); }
}

// Order.php
class Order extends Model {
    protected $fillable = [
        'order_number','shipping_method_id','coupon_id',
        'first_name','last_name','phone','city','quartier','zip_code','address_line',
        'status','subtotal','shipping_cost','discount_amount','total','notes',
    ];

    protected $casts = [
        'subtotal'        => 'decimal:2',
        'shipping_cost'   => 'decimal:2',
        'discount_amount' => 'decimal:2',
        'total'           => 'decimal:2',
    ];

    protected static function booted(): void {
        static::creating(function (Order $o) {
            $o->order_number ??= 'LX-' . random_int(1000, 9999) . '-' . strtoupper(Str::random(3));
        });
    }

    public function items():           HasMany   { return $this->hasMany(OrderItem::class); }
    public function statusHistories(): HasMany   { return $this->hasMany(OrderStatusHistory::class)->orderBy('crea
ted_at'); }
    public function shippingMethod():  BelongsTo { return $this->belongsTo(ShippingMethod::class); }
    public function coupon():          BelongsTo { return $this->belongsTo(Coupon::class); }
}
```

3.4 Service Contracts

`OrderService::create(StoreOrderRequest $request): Order`

Execution sequence — all within a single `DB::transaction()`:

1. Resolve each `product_id` + `size_label` from DB. Throw `ProductNotFoundException` if any missing or `is_active = false`.
2. For each item: verify requested `quantity` ≤ `product_sizes.stock` (or `products.stock` if no size). Throw `StockInsufficientException` if any fail.
3. Resolve prices from DB: `unit_price = product.price + (size.price_modifier ?? 0)`. Ignore any client-sent prices.
4. Calculate `subtotal = Σ(unit_price × quantity)`.
5. Resolve `shipping_cost` from `shipping_methods.price` for the requested `shipping_method_id`.
6. If `coupon_code` present: call `CouponService::apply($code, $subtotal)`. Returns `discount_amount`. Increments `used_count`. Throws on invalid/expired.
7. Calculate `total = subtotal + shipping_cost - discount_amount`.
8. Create `Order` record.
9. Create `OrderItem` records.
10. Decrement `stock` on each `ProductSize` (or `Product` if no sizes).
11. Create initial `OrderStatusHistory`: `{ status: confirmed, label: 'Order Valid', location: null }`.
12. Dispatch `SendOrderConfirmationSms` to queue.
13. Return `Order` with loaded relations.

`CouponService::validate(string $code, float $subtotal): array`

Does not increment `used_count`. Returns:

```
['valid' => bool, 'discount_amount' => float, 'message' => string, 'code_error' => string|null]
```

`CouponService::apply(string $code, float $subtotal): float`

Called by `OrderService` only. Increments `used_count`. Returns `discount_amount`.

`ProductSearchService::search(array $filters): LengthAwarePaginator`

Builds Eloquent query. Always eager loads `brand:id,name,slug`, `category:id,name,slug`, `images` (primary only). Applies:

- `where('is_active', true)` — always
- `where('brand_id', ...)`, `where('category_id', ...)`, `where('gender', ...)` — when present
- `whereBetween('price', [...])` — when `price_min` / `price_max` present
- `havingRaw('avg_rating >= ?', [...])` via subquery — when `min_rating` present
- `where('original_price', '!=', null)` — when `on_sale = true`
- `whereFullText(['name','description'], $search)` — when `search` present (MySQL FULLTEXT)
- Sort: `orderBy('price', 'asc'|'desc')`, `orderBy('created_at', 'desc')`, or by avg_rating subquery

3.5 Form Request Contracts

`StoreOrderRequest`:

```
'first_name'         => 'required|string|max:100',
'last_name'          => 'required|string|max:100',
'phone'              => 'required|string|regex:/^\+?[0-9]{8,15}$/',
'city'               => 'required|string|max:100',
'quartier'           => 'nullable|string|max:100',
'zip_code'           => 'nullable|string|max:20',
'address_line'       => 'required|string|max:500',
'shipping_method_id' => 'required|integer|exists:shipping_methods,id',
'coupon_code'        => 'nullable|string|max:50',
'items'              => 'required|array|min:1|max:50',
'items.*.product_id' => 'required|integer|exists:products,id',
'items.*.size_label' => 'nullable|string|max:50',
'items.*.quantity'   => 'required|integer|min:1|max:100',
```

`StoreReviewRequest` :

```
'product_id'    => 'required|integer|exists:products,id',
'order_number'  => 'nullable|string|max:20',
'reviewer_name' => 'required|string|max:100',
'rating'        => 'required|integer|min:1|max:5',
'body'          => 'nullable|string|max:2000',
'images'        => 'nullable|array|max:3',
'images.*'      => 'file|mimes:jpeg,png,webp|max:5120',
```

`ValidateCouponRequest` :

```
'code'     => 'required|string|max:50',
'subtotal' => 'required|numeric|min:0',
```

3.6 Resource Output Contracts

`ProductResource` (listing — lean):

```
{
  "id": 1,
  "name": "SUGAR POP",
  "slug": "sugar-pop-abc123",
  "subtitle": "Body Butter",
  "price": "140.00",
  "original_price": "200.00",
  "rating": 4.9,
  "review_count": 180,
  "is_featured": true,
  "gender": "woman",
  "brand": { "id": 2, "name": "Boss", "slug": "boss" },
  "category": { "id": 3, "name": "Corps", "slug": "corps" },
  "primary_image": "https://cdn.bloomparfums.ma/products/sugar-pop.webp"
}
```

`ProductDetailResource` (detail — full): Adds: `description` , `ingredients` , `sizes[]` , `images[]` (full gallery), `latest_reviews[]` (4 items, approved only).

`OrderTrackResource` (public — redacted): Exposes: `order_number` , `status` , `status_histories[]` , `items_count` . Never exposes: `phone` , `address_line` , `total` , `discount_amount` , `coupon_id` .

`CouponValidationResource` :

```
{
  "valid": true,
  "code": "PROMO10",
  "discount_type": "fixed",
  "discount_value": 40.00,
  "discount_amount": 40.00,
  "message": "Coupon validé — vous économisez 40 DH"
}
```

3.7 Caching Strategy

```
// Read-heavy catalog data — invalidated by admin mutations

Cache::remember('products.featured', 3600, fn() =>
    Product::with(['brand:id,name,slug','category:id,name,slug','images'])
           ->where('is_active', true)
           ->where('is_featured', true)
           ->limit(8)
           ->get()
);

Cache::tags(['brands'])->remember('brands.all', 21600, fn() => Brand::all());

Cache::tags(['categories'])->remember('categories.tree', 21600, fn() =>
    Category::with('children')->whereNull('parent_id')->orderBy('sort_order')->get()
);

Cache::tags(['products'])->remember("product.{$slug}", 1800, fn() =>
    Product::with(['brand','category','images','sizes'])
           ->where('slug', $slug)
           ->where('is_active', true)
           ->firstOrFail()
);

Cache::tags(['reviews'])->remember("reviews.{$slug}", 900, fn() =>
    Review::with('images')
          ->where('product_id', $product->id)
          ->where('is_approved', true)
          ->latest()
          ->paginate(10)
);
```

Cache invalidation in admin controllers:

```
// On any product mutation:
Cache::tags(['products'])->flush();
Cache::forget("product.{$slug}");
Cache::forget('products.featured');
```

4. Admin Authentication Model

4.1 Stack Decision

Laravel Sanctum — stateful SPA mode. Token issued by backend, stored in `HttpOnly, Secure, SameSite=Strict` cookie. JavaScript on the frontend never reads the token. `withCredentials: true` on Axios sends the cookie automatically on every request.

No JWT. No Bearer headers. No localStorage. No `js-cookie` writes.

4.2 Admin Model

`Admin extends Authenticatable`. Uses `HasApiTokens` from Sanctum. The `guard_name` in `config/auth.php` is `admin`. Tokens are stored in `personal_access_tokens` (Sanctum default table).

```
// config/auth.php
'guards' => [
    'web'   => ['driver' => 'session', 'provider' => 'users'],
    'admin' => ['driver' => 'sanctum',  'provider' => 'admins'],
],

'providers' => [
    'admins' => ['driver' => 'eloquent', 'model' => App\Models\Admin::class],
],
```

No `users` guard. No `customers` guard. The `users` provider is removed.

4.3 Auth Flow

Login sequence:

1. `POST /api/v1/admin/auth/login` receives `{email, password}`.
2. `AdminAuthController` validates via `Auth::guard('admin')->attempt(...)`.
3. On success: `$admin->createToken('admin-session')`.
4. Response sets `Set-Cookie: admin_token={token}; HttpOnly; Secure; SameSite=Strict; Max-Age=86400; Path=/`.
5. Response body: `{ "message": "Authenticated", "admin": { "id": 1, "email": "..." } }`.

Per-request auth:

1. Browser sends `admin_token` cookie automatically (Sanctum stateful).
2. `auth:sanctum` middleware validates the token.
3. `EnsureAdmin` middleware verifies `$request->user() instanceof Admin`.

Logout:

1. `POST /api/v1/admin/auth/logout`.
2. `$request->user()->currentAccessToken()->delete()`.
3. Response clears cookie: `Set-Cookie: admin_token=; Max-Age=0; Path=/`.

4.4 Middleware Stack

```
routes/api.php

// Public routes — no middleware except ForceJsonResponse (global)
Route::prefix('v1')->group(function () {
    Route::get('/products', [ProductController::class, 'index']);
    Route::get('/products/{slug}', [ProductController::class, 'show']);
    Route::get('/products/validate', [ProductController::class, 'validateIds']);
    Route::get('/brands', [BrandController::class, 'index']);
    Route::get('/categories', [CategoryController::class, 'index']);
    Route::get('/shipping-methods', [ShippingMethodController::class, 'index']);
    Route::post('/cart/coupon/validate', [CouponController::class, 'validate']);
    Route::post('/orders', [OrderController::class, 'store']);
    Route::get('/orders/{orderNumber}/track', [OrderController::class, 'track']);
    Route::post('/reviews', [ReviewController::class, 'store']);
    Route::get('/products/{slug}/reviews', [ReviewController::class, 'index']);
});

// Admin routes — Sanctum + EnsureAdmin + throttle
Route::prefix('v1/admin')->middleware(['auth:sanctum', 'ensure.admin', 'throttle:300,1'])->group(function () {
    Route::get('/auth/me', [AdminAuthController::class, 'me']);
    Route::post('/auth/logout', [AdminAuthController::class, 'logout']);

    Route::apiResource('/products',   Admin\ProductController::class);
    Route::apiResource('/brands',     Admin\BrandController::class);
    Route::apiResource('/categories', Admin\CategoryController::class);
    Route::apiResource('/coupons',    Admin\CouponController::class);
    Route::get('/orders',             [Admin\OrderController::class, 'index']);
    Route::get('/orders/{id}',        [Admin\OrderController::class, 'show']);
    Route::patch('/orders/{id}/status', [Admin\OrderController::class, 'updateStatus']);
    Route::get('/reviews',            [Admin\ReviewController::class, 'index']);
    Route::patch('/reviews/{id}/approve', [Admin\ReviewController::class, 'approve']);
});

// Login — outside auth middleware
Route::post('/v1/admin/auth/login', [AdminAuthController::class, 'login'])
     ->middleware('throttle:5,1');
```

4.5 `EnsureAdmin` Middleware

```
class EnsureAdmin {
    public function handle(Request $request, Closure $next): Response {
        if (! $request->user() instanceof \App\Models\Admin) {
            return response()->json(['message' => 'Forbidden.'], 403);
        }
        return $next($request);
    }
}
```

Registered in `bootstrap/app.php` as `'ensure.admin'`.

4.6 Roles

There are no admin roles. Every authenticated admin has full access to every admin endpoint. The system has one admin. Role-based access control is not a requirement and will not be built.

---

5. Wishlist Cookie System

5.1 Specification

Property	Value
Cookie name	`bloom_wishlist`
Storage format	JSON array of integer product IDs
Max size	50 items
Expiration	30 days from last modification
Scope	`Path=/`, `SameSite=Lax`
HttpOnly	`false` — frontend must read/write it
Secure	`true` in production, `false` in local
Example value	`[12, 47, 103, 8]`

`SameSite=Lax` (not Strict) is required because the user may navigate from an external link (email, social) and the cookie must be readable on first load.

`HttpOnly=false` is intentional and justified: the wishlist contains no sensitive data — only public product IDs. Making it HttpOnly would require all wishlist operations to go through the backend, adding latency to a client-only concern.

5.2 Frontend Cookie Utility — Complete Specification

The following module is the single canonical wishlist utility. Located at `frontend/lib/wishlist.ts`.

```
import Cookies from 'js-cookie';

const COOKIE_NAME    = 'bloom_wishlist';
const EXPIRY_DAYS    = 30;
const MAX_ITEMS      = 50;
const COOKIE_OPTIONS = {
  expires : EXPIRY_DAYS,
  path    : '/',
  sameSite: 'Lax' as const,
  secure  : process.env.NODE_ENV === 'production',
};

export function getWishlist(): number[] {
  try {
    const raw = Cookies.get(COOKIE_NAME);
    if (!raw) return [];
    const parsed = JSON.parse(raw);
    return Array.isArray(parsed) ? parsed.filter(Number.isInteger) : [];
  } catch {
    return [];
  }
}

export function addToWishlist(productId: number): void {
  const current = getWishlist();
  if (current.includes(productId)) return;
  if (current.length >= MAX_ITEMS) return;
  const updated = [...current, productId];
  Cookies.set(COOKIE_NAME, JSON.stringify(updated), COOKIE_OPTIONS);
}

export function removeFromWishlist(productId: number): void {
  const updated = getWishlist().filter(id => id !== productId);
  if (updated.length === 0) {
    Cookies.remove(COOKIE_NAME, { path: '/' });
    return;
  }
  Cookies.set(COOKIE_NAME, JSON.stringify(updated), COOKIE_OPTIONS);
}

export function isInWishlist(productId: number): boolean {
  return getWishlist().includes(productId);
}

export function clearWishlist(): void {
  Cookies.remove(COOKIE_NAME, { path: '/' });
}
```

5.3 Wishlist Lifecycle

```
User clicks heart on ProductCard
    → isInWishlist(productId) → boolean
    → if false: addToWishlist(productId) — write cookie
    → if true:  removeFromWishlist(productId) — update cookie

User navigates to /wishlist
    → getWishlist() → number[]
    → if empty: render empty state
    → if non-empty: GET /api/v1/products/validate?ids=12,47,103
        → backend resolves IDs to current product state
        → inactive/deleted products returned as { active: false }
        → frontend filters them out OR marks them as "unavailable"
        → renders product grid from resolved data

Cookie expires after 30 days of inactivity
    → getWishlist() returns []
    → /wishlist renders empty state
    → no backend call needed
```

5.4 Backend Role in Wishlist System

The backend performs exactly one operation related to the wishlist:

```
GET /api/v1/products/validate?ids=12,47,103
```

It resolves a comma-separated list of product IDs into current product state. It performs no writes. It stores no wishlist. It has no knowledge of which user owns which list. The endpoint is public and rate-limited.

If a product in the wishlist was deleted (soft-delete), the backend returns `{ id: X, active: false }`. The frontend is responsible for handling this gracefully (remove from cookie, show "product unavailable" label, or silently exclude).

---

6. API Contract

6.1 Global Rules

- All responses are JSON (`Content-Type: application/json`).
- All error responses use the structure: `{ "message": "...", "code": "ERROR_CODE", "errors": {} }`.
- `errors` is present only on `422` validation failures, containing field-level messages.
- HTTP status codes are semantic — no `200` for errors.
- All collections are wrapped in `{ "data": [...], "meta": {...}, "links": {...} }`.
- All single resources are wrapped in `{ "data": {...} }`.
- Timestamps in ISO 8601 UTC.

6.2 Public Endpoints

`GET /api/v1/products`

Auth: None — **Rate limit:** 120/min

Query params:

```
brand_id     integer   optional
category_id  integer   optional
gender       enum      optional  man|woman|child|unisex
price_min    numeric   optional  default: 0
price_max    numeric   optional
min_rating   numeric   optional  1-5
on_sale      boolean   optional  true = original_price IS NOT NULL
search       string    optional  min 3 chars, max 100
sort         enum      optional  newest|price_asc|price_desc|rating
page         integer   default: 1
per_page     integer   default: 20, max: 60
```

Response `200`: Paginated `ProductResource` collection.

`GET /api/v1/products/{slug}`

Auth: None — **Cache:** Redis 30min — **Rate limit:** 120/min

Response `200`: `ProductDetailResource`
Response `404`: `{ "message": "Product not found.", "code": "PRODUCT_NOT_FOUND" }`
Response `410`: `{ "message": "Product unavailable.", "code": "PRODUCT_UNAVAILABLE" }` — when `is_active = false`

`GET /api/v1/products/validate`

Auth: None — **Rate limit:** 60/min

Query: `?ids=12,47,103` (comma-separated integers, max 50)

Response `200`:

```
{
  "data": [
    { "id": 12,  "active": true,  "name": "SUGAR POP", "slug": "sugar-pop-abc123", "price": "140.00", "primary_ima
ge": "..." },
    { "id": 47,  "active": false, "name": null, "slug": null, "price": null, "primary_image": null },
    { "id": 103, "active": true,  "name": "OVER DOSE",  "slug": "over-dose-xyz789", "price": "100.00", "primary_im
age": "..." }
  ]
}
```

`GET /api/v1/brands`

Auth: None — **Cache:** Redis 6h

Response `200`:

```
{ "data": [{ "id": 1, "name": "Boss", "slug": "boss", "logo_url": "...", "product_count": 12 }] }
```

`GET /api/v1/categories`

Auth: None — **Cache:** Redis 6h

Response `200`: Nested tree. Root categories with `children[]` array.

`GET /api/v1/shipping-methods`

Auth: None — **Cache:** Redis 24h

Response `200`:

```
{
  "data": [
    { "id": 1, "name": "Free Shipping",  "description": "Laayoune",                   "price": "0.00",  "free_ove
r": null },
    { "id": 2, "name": "Région",         "description": "Laayoune-Sakia el Hamra",    "price": "20.00", "free_ove
r": null },
    { "id": 3, "name": "National",       "description": "Tous les villes du Maroc",   "price": "35.00", "free_ove
r": "590.00" }
  ]
}
```

`POST /api/v1/cart/coupon/validate`

Auth: None — **Rate limit:** 20/min

Request: `{ "code": "PROMO10", "subtotal": 760.00 }`

Response `200` (valid): Full `CouponValidationResource`.
Response `200` (invalid): `{ "data": { "valid": false, "code": "COUPON_EXPIRED", "message": "This coupon has expired." } }`

This endpoint always returns `200`. The validity is indicated by `data.valid`. Never returns `4xx` for an invalid coupon — that would break the UX. `4xx` is reserved for malformed requests.

`POST /api/v1/orders`

Auth: None — **Rate limit:** 10/min

Request body: See `StoreOrderRequest` contract in Section 3.5.

Response `201`: `OrderResource` including `order_number`, `status`, financials, and resolved `items[]`.

Errors:

- `422`: Validation failure (missing fields, invalid product_id, invalid shipping_method_id)
- `409`: `{ "message": "Insufficient stock for SUGAR POP (50ml).", "code": "STOCK_INSUFFICIENT", "product_id": 1 }`
- `410`: `{ "message": "Coupon PROMO10 has expired.", "code": "COUPON_EXPIRED" }`
- `422`: `{ "message": "Minimum order amount for this coupon is 500 DH.", "code": "COUPON_MIN_ORDER" }`

`GET /api/v1/orders/{orderNumber}/track`

Auth: None — **Rate limit:** 10/min

Query: `?phone=+212611955060` (required)

Backend validates that the `phone` matches `orders.phone` for the given `order_number`. Same `404` is returned whether the order doesn't exist or the phone doesn't match — prevents order enumeration.

Response `200`: `OrderTrackResource` (redacted — no address, no total, no coupon).
Response `404`: `{ "message": "Order not found.", "code": "ORDER_NOT_FOUND" }` — always, regardless of reason.

`POST /api/v1/reviews`

Auth: None — **Rate limit:** 5/min — **Content-Type:** `multipart/form-data`

Fields: See `StoreReviewRequest` contract in Section 3.5.

Response `201`:

```
{
  "data": {
    "id": 42,
    "reviewer_name": "Ayoub L.",
    "rating": 5,
    "body": "Excellent fragrance, long-lasting.",
    "is_approved": false,
    "message": "Your review has been received and will be published after moderation."
  }
}
```

`GET /api/v1/products/{slug}/reviews`

Auth: None — **Cache:** Redis 15min

Response `200`:

```
{
  "data": [
    { "id": 1, "reviewer_name": "Zineb E.", "rating": 5, "body": "...", "created_at": "2024-05-16T00:00:00Z", "ima
ges": [] }
  ],
  "meta": { "current_page": 1, "last_page": 3, "average_rating": 4.9, "total_reviews": 180
  }
}
```

6.3 Admin Endpoints

All require `admin_token` cookie. Sanctum validates. `EnsureAdmin` middleware confirms `Admin` instance. Throttle: 300/min.

`POST /api/v1/admin/auth/login`

Auth: None — **Throttle:** 5/min
Request: `{ "email": "admin@bloomparfums.ma", "password": "..." }`
Response `200`: `{ "message": "Authenticated", "admin": { "id": 1, "email": "..." } }` + `Set-Cookie: admin_token=...; HttpOnly; Secure; SameSite=Strict; Max-Age=86400; Path=/`
Response `401`: `{ "message": "Invalid credentials.", "code": "AUTH_FAILED" }`
Response `429`: After 5 failed attempts within 1 minute.

`POST /api/v1/admin/auth/logout`

Auth: Required
Response `200`: `{ "message": "Logged out." }` + `Set-Cookie: admin_token=; Max-Age=0; Path=/`

`GET /api/v1/admin/auth/me`

Auth: Required
Response `200`: `{ "data": { "id": 1, "email": "admin@bloomparfums.ma" } }`

`GET|POST|PUT|DELETE /api/v1/admin/products`

Full CRUD. `POST` and `PUT` accept `multipart/form-data` for image uploads.
Cache invalidation on every mutation: `Cache::tags(['products'])->flush()`.

`GET|POST|PUT|DELETE /api/v1/admin/brands`

Full CRUD. Cache invalidation: `Cache::tags(['brands'])->flush()`.

`GET|POST|PUT|DELETE /api/v1/admin/categories`

Full CRUD. Cache invalidation: `Cache::tags(['categories'])->flush()`.

`GET|POST|PUT|DELETE /api/v1/admin/coupons`

Full CRUD.

`GET /api/v1/admin/orders`

Paginated. Filter params: `status`, `date_from`, `date_to`, `search` (order_number or phone).

`GET /api/v1/admin/orders/{id}`

Full order including `items[]`, `statusHistories[]`, `shippingMethod`, `coupon`.

`PATCH /api/v1/admin/orders/{id}/status`

Request: `{ "status": "shipped", "label": "Shipped via CTM", "location": "Casablanca" }`
Action: Updates `orders.status`. Appends row to `order_status_histories`.
Response `200`: Updated `OrderResource`.

`GET /api/v1/admin/reviews`

Paginated. Filter: `is_approved` (boolean), `product_id` (integer).

`PATCH /api/v1/admin/reviews/{id}/approve`

Request: `{ "approved": true }`
Action: Sets `is_approved`, sets/clears `approved_at`. Fires `Cache::tags(['reviews'])->flush()`.
Response `200`: `ReviewResource`.

6.4 Removed/Replaced Endpoints

Endpoint	Action	Replacement
`POST /api/v1/auth/register`	**Removed**	No equivalent
`POST /api/v1/auth/login`	**Replaced**	`POST /api/v1/admin/auth/login`
`POST /api/v1/auth/logout`	**Replaced**	`POST /api/v1/admin/auth/logout`
`POST /api/v1/auth/refresh`	**Removed**	Sanctum tokens do not refresh; expired = re-login
`GET /api/v1/auth/me`	**Replaced**	`GET /api/v1/admin/auth/me`
`GET	**Removed**	POST /api/v1/users`

---

7. Frontend Integration Rules

7.1 Data Fetching Strategy

Page / Component	Strategy	Reason
`/` — Homepage, BestSellers	`getStaticProps` (SSG) + `revalidate: 3600`	Public, stable data. No auth. SEO critical.
`/product/[slug]`	`generateStaticParams` + `revalidate: 1800`	SEO-critical product pages. Admin mutations trigger revalidation via On-Demand ISR.
`/collection`	SWR client-side	Filter state changes require re-fetching. URL query params drive SWR key.
`/wishlist`	SWR client-side	Resolved from cookie on mount. Cookie read is synchronous; API call is async.
`/checkout`	CSR — static data via SWR	Shipping methods fetched once on mount. Cart from Zustand.
`/track-order`	CSR — form submit	No data pre-fetch. Field input → API call on submit.
`/order-status`	SWR with `refreshInterval: 30000`	Poll for real-time status updates without full page reload.

Page / Component	Strategy	Reason
`/feedback`	CSR — from query param	Order number passed as URL param; products resolved from order API.
`/admin/*`	CSR — Axios	Admin panel never uses SSR. All admin data fetched client-side after auth check.

7.2 Cart — Zustand Store Contract

```
// store/cart.ts
interface CartItem {
  productId   : number;
  productName : string;
  slug        : string;
  sizeLabel   : string | null;
  quantity    : number;
  unitPrice   : number;
  imageUrl    : string;
}

interface CartStore {
  items       : CartItem[];
  addItem     : (item: CartItem) => void;
  removeItem  : (productId: number, sizeLabel: string | null) => void;
  updateQty   : (productId: number, sizeLabel: string | null, qty: number) => void;
  clearCart   : () => void;
  itemCount   : () => number;
  subtotal    : () => number;
}
```

The cart is never sent to the backend as a pre-existing entity. At checkout, the cart contents are serialized as `items[]` in `POST /api/v1/orders`. The backend re-resolves all prices. The client subtotal display is UX-only.

7.3 Admin Auth Flow — Next.js Side

Admin login page: `/admin/login` — CSR. On submit: `POST /api/v1/admin/auth/login` with `withCredentials: true`. On `200`: `router.push('/admin/dashboard')`. On `401`: display error inline.

Admin route protection: `middleware.ts` at project root:

```
import { NextRequest, NextResponse } from 'next/server';

export function middleware(request: NextRequest) {
  const isAdminRoute   = request.nextUrl.pathname.startsWith('/admin');
  const isLoginRoute   = request.nextUrl.pathname === '/admin/login';
  const hasAdminCookie = request.cookies.has('admin_token');

  if (isAdminRoute && !isLoginRoute && !hasAdminCookie) {
    return NextResponse.redirect(new URL('/admin/login', request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/admin/:path*'],
};
```

The presence check of `admin_token` cookie is sufficient for the middleware redirect gate. The actual token validity is enforced by the backend on every API call — an expired or invalid cookie will produce a `401`, which the Axios response interceptor handles by redirecting to `/admin/login`.

7.4 Axios Instance — Final Configuration

```
// services/api.ts — final revision
import axios from 'axios';

const api = axios.create({
  baseURL        : process.env.NEXT_PUBLIC_API_URL,
  withCredentials: true,        // sends Sanctum admin_token cookie automatically
  headers        : { 'Accept': 'application/json' },
});

// No request interceptor that reads js-cookie.
// No manual Authorization header.
// Sanctum cookie is sent automatically by the browser.

api.interceptors.response.use(
  response => response,
  async error => {
    if (error.response?.status === 401) {
      if (typeof window !== 'undefined') {
        window.location.href = '/admin/login';
      }
    }
    return Promise.reject(error);
  }
);

export default api;
```

The previous Bearer token interceptor is completely removed. The `401` handler does not retry — Sanctum tokens do not refresh. Expiry triggers re-login.

7.5 Wishlist Integration in Components

`ProductCard` integrates as follows:

```
import { addToWishlist, removeFromWishlist, isInWishlist } from '@/lib/wishlist';

// Inside component:
const [wished, setWished] = useState(() => isInWishlist(product.id));

function toggleWishlist(e: React.MouseEvent) {
  e.preventDefault();
  e.stopPropagation();
  if (wished) {
    removeFromWishlist(product.id);
  } else {
    addToWishlist(product.id);
  }
  setWished(prev => !prev);
}
```

The current `useState(false)` local state pattern is replaced by initializing from the cookie. The cookie is checked synchronously on component mount. No API call occurs at toggle time.

7.6 Cookie Wishlist on `/wishlist` Page

```
// app/wishlist/page.tsx
'use client'
import useSWR from 'swr';
import { getWishlist } from '@/lib/wishlist';

const ids = getWishlist(); // synchronous, no API call

const { data, isLoading } = useSWR(
  ids.length > 0 ? `/api/v1/products/validate?ids=${ids.join(',')}` : null,
  fetcherFn,
  { revalidateOnFocus: false }
);

const activeProducts = data?.data.filter((p: any) => p.active) ?? [];
```

When `ids.length === 0`, SWR key is `null` — no request is made. Empty state is rendered directly.

7.7 Removed Frontend Artifacts

Artifact	Action
`app/register/page.tsx`	Delete
`app/login/page.tsx`	Repurpose or delete — replaced by `app/admin/login/page.tsx`
`app/dashboard/page.tsx` + `DashboardClient.tsx`	Delete — replaced by `app/admin/dashboard/`
`lib/auth.ts` `setAuthToken`, `getAuthToken`, `clearAuthToken`	Delete entirely
`lib/auth.ts` `isAuthenticated`	Delete
`lib/auth.ts` `serverFetch`	Delete — admin panel uses client-side Axios
Bearer token request interceptor in `services/api.ts`	Delete
`authService.login`, `authService.register` in `services/api.ts`	Delete
`components/Navbar.tsx` (DashboardClient nav)	Delete — admin panel gets its own nav

8. Security & Risk Mitigation

8.1 Cookie Tampering — Wishlist

Threat: Attacker manually sets `bloom_wishlist=["'; DROP TABLE products; --"]` or a non-integer array.

Mitigation: The frontend utility `getWishlist()` wraps the JSON parse in `try/catch` and filters with `Array.isArray(parsed) ? parsed.filter(Number.isInteger) : []`. Non-integer values are silently discarded. The backend `GET /api/v1/products/validate` accepts only integers and validates each with `exists:products,id`. A tampered cookie results in an empty or partially resolved wishlist — no security consequence.

8.2 Product Deleted While in Wishlist

Scenario: Product ID 47 is in the user's cookie. Admin soft-deletes product 47.

Resolution path:

1. `GET /api/v1/products/validate?ids=12,47` is called when the user opens `/wishlist`.
2. Backend: `Product::withoutTrashed()->whereIn('id', [12, 47])->where('is_active', true)` — ID 47 is excluded.
3. Response: `{ id: 47, active: false, name: null, ... }`.
4. Frontend: filters products where `active === false`, optionally removes them from the cookie via `removeFromWishlist(47)`, renders "Product no longer available" or silently excludes.
5. Cookie is self-healing — on the next wishlist view, the stale ID is cleaned up.

8.3 Admin Token Expiration

Sanctum default token TTL: Configured in `config/sanctum.php` as `expiration` (minutes). Set to `1440` (24 hours) for production.

On expiry: Axios `401` interceptor catches the response → `window.location.href = '/admin/login'`. No retry. No token refresh. Admin re-authenticates.

The Next.js `middleware.ts` cookie presence check is a redirect gate only — it does not validate token freshness. The backend is the authoritative validator.

8.4 Admin Login Brute-Force

Laravel throttle middleware on the login route: `throttle:5,1` (5 requests per 1 minute per IP). After 5 failures: `429 Too Many Requests` for 60 seconds.

Extended block: Custom Redis-backed middleware increments `admin_login_fail:{ip}` on each `401`. After 10 cumulative failures within 1 hour: block for 30 minutes regardless of rate limit window. Implemented in `AdminAuthController` using `Cache::increment` + `Cache::put` with expiry.

8.5 Order Enumeration Prevention

`GET /api/v1/orders/{orderNumber}/track` requires `?phone=...`. Backend performs:

```
$order = Order::where('order_number', $orderNumber)
              ->where('phone', $request->phone)
              ->first();

if (!$order) {
    return response()->json(['message' => 'Order not found.', 'code' => 'ORDER_NOT_FOUND'], 404);
}
```

A wrong phone for a valid order number returns identical `404` to a non-existent order number. An attacker cannot determine whether an order number exists.

Additionally, rate limiting at 10 req/min prevents systematic phone enumeration against known order numbers.

8.6 Price Integrity

Frontend prices are **never trusted**. `OrderService` resolves all prices from the database:

```
$unitPrice = $product->price + ($size?->price_modifier ?? 0.00);
```

Any client-sent `price` field in the order request body is ignored entirely. The `StoreOrderRequest` does not have a `price` field. An attacker who modifies the request body to send `"price": 1` will have no effect — the price is computed server-side from the product record.

8.7 Stock Race Condition

Two simultaneous orders for the last item of a product can both pass the stock check before either decrements. Mitigation: pessimistic locking within the `DB::transaction()`:

```
DB::transaction(function () use ($items) {
    foreach ($items as $item) {
        $size = ProductSize::where('product_id', $item['product_id'])
                           ->where('label', $item['size_label'])
                           ->lockForUpdate()     // SELECT ... FOR UPDATE
                           ->first();

        if ($size->stock < $item['quantity']) {
            throw new StockInsufficientException($product->name, $item['size_label']);
        }

        $size->decrement('stock', $item['quantity']);
    }
});
```

`lockForUpdate()` acquires a row-level exclusive lock for the transaction duration. Competing transactions block until the lock is released.

8.8 Image Upload Security

1. MIME validation from file content via `mimes:jpeg,png,webp` (Laravel reads the first bytes, not the extension).
2. Max file size: 5120 KB (5 MB) enforced in `StoreReviewRequest`.
3. Max 3 images per review.
4. Files are never stored in `public/` — only `Storage::disk('s3')`.
5. S3 bucket policy: no public `s3:GetObject`. Files served exclusively through Cloudflare CDN with signed URLs or public CDN origin pull.
6. Processing is asynchronous — `ProcessReviewImage` job resizes to 800px wide, converts to WebP, then uploads. The original file is discarded after processing.

8.9 CORS Configuration

```
// config/cors.php
'allowed_origins' => [
    env('FRONTEND_URL', 'http://localhost:3000'),
],
'allowed_methods'   => ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
'allowed_headers'   => ['Content-Type', 'Accept', 'X-Requested-With'],
'exposed_headers'   => [],
'max_age'           => 0,
'supports_credentials' => true,  // required for Sanctum cookie
```

`supports_credentials: true` is required for `withCredentials: true` on Axios. With this set, `allowed_origins` must list explicit origins — wildcard `*` is rejected by the browser for credentialed requests.

Production config: `allowed_origins: ['https://bloomparfums.ma', 'https://www.bloomparfums.ma']`.

8.10 Security Headers

`SecurityHeaders` middleware applied globally in `bootstrap/app.php`:

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
```

`Content-Security-Policy` is not set server-side to avoid conflict with Next.js CSP configuration, which is managed in `next.config.mjs` headers.

8.11 Environment Separation

Variable	local	production
`APP_DEBUG`	`true`	`false`
`APP_ENV`	`local`	`production`
`CACHE_DRIVER`	`file`	`redis`
`QUEUE_CONNECTION`	`sync`	`redis`
`SESSION_SECURE_COOKIE`	`false`	`true`
`SANCTUM_STATEFUL_DOMAINS`	`localhost:3000`	`bloomparfums.ma`
`FRONTEND_URL`	`http://localhost:3000`	`https://bloomparfums.ma`
`AWS_BUCKET`	(local public disk)	`bloom-parfums-media`

`APP_DEBUG=true` on production exposes environment variables and full stack traces in API error responses. This is a critical vulnerability. A production deployment pipeline must assert `APP_DEBUG=false`.

9. Final Technical Notes

9.1 What Is NOT Built

The following are explicitly out of scope and will not be architected:

- Customer accounts or any public auth flow
- Loyalty program or points system
- Wishlist persistence in database
- Cart persistence (no cart table, no sessions)
- Real-time inventory WebSockets (polling on `/order-status` is sufficient)
- Multi-admin roles or permissions
- Admin password reset UI (use Artisan seeder)
- Multi-language (i18n) — all content is French/Moroccan
- Payment gateway integration — cash-on-delivery only (no PCI-DSS scope)

9.2 Implementation Sequence

Phase	Deliverables	Blocks
1 — Backend Foundation	Drop `users` migration. Create 13 migrations. Seed brands/categories/shipping. `Admin` model + Sanctum install. `.env` config.	Everything
2 — Public Catalog API	`ProductController`, `BrandController`, `CategoryController`, `ShippingMethodController`. Redis caching. `ProductResource`, `ProductDetailResource`.	Frontend wiring
3 — Frontend Catalog Wiring	SWR in `/collection`, `/product/[slug]`, `BestSellers`. Zustand cart store. `lib/wishlist.ts`. Remove hardcoded data.	Cart, Checkout

Phase	Deliverables	Blocks
4 — Order System	`OrderService`, `StoreOrderRequest`, `POST /api/v1/orders`, `GET /orders/track`. Wire Checkout submit, Success page, TrackOrder form, OrderStatus SWR poll.	Coupon, Review
5 — Coupon System	`CouponService::validate`, `POST /cart/coupon/validate`. Wire CartDrawer + Checkout coupon input.	Order discount
6 — Review System	`ReviewService`, `POST /api/v1/reviews`, `GET /products/{slug}/reviews`. Wire ReviewModal submit. Wire Feedback page product list from order.	Admin moderation
7 — Admin Panel	`/admin/login`, `/admin/dashboard`, product CRUD, order status updates, review moderation. `middleware.ts` protection.	Production
8 — Production Hardening	Rate limits, Redis verified, S3 uploads, `SendOrderConfirmationSms` queue, security headers, `APP_DEBUG=false`, `SANCTUM_STATEFUL_DOMAINS` set.	Launch

9.3 Sanctum Installation Checklist

```
# 1. Remove JWT
composer remove tymon/jwt-auth

# 2. Install Sanctum
composer require laravel/sanctum

# 3. Publish config
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"

# 4. Run migration (creates personal_access_tokens table)
php artisan migrate

# 5. Add HasApiTokens to Admin model
# 6. Configure guard in config/auth.php
# 7. Add EnsureAdmin middleware to bootstrap/app.php
# 8. Set SANCTUM_STATEFUL_DOMAINS in .env
```

9.4 Critical Fixes — Pre-Implementation

These must be resolved before any Phase 1 work begins:

Fix	File	Change
Remove `Math.random()` from histogram	`app/collection/page.tsx`	Replace with fixed array: `[40, 65, 80, 90, 70, 55, 45, 30, 60, 75]`
Define missing CSS classes	`app/globals.css`	Add `form-input`, `btn-secondary` with design system tokens
`brand-600` / `brand-50` Tailwind tokens	`tailwind.config.ts`	Add `brand` color scale or replace selectors
`<Link>` submit in track-order	`app/track-order/page.tsx`	Replace with `<form onSubmit={handleSubmit}>`
Remove `app/register/page.tsx`	filesystem	`rm frontend/app/register/page.tsx`
Remove public auth from `routes/api.php`	`backend/routes/api.php`	Delete `/auth/register`, `/auth/login`, `/auth/refresh`, `/auth/me` public routes

9.5 N+1 Query Checklist

These eager loads are mandatory on every query — not optional optimizations:

```
// Product listings (any paginated collection)
Product::with([
    'brand:id,name,slug',
    'category:id,name,slug',
    'images' => fn($q) => $q->where('is_primary', true)->select(['id','product_id','url','alt']),
])->where('is_active', true)->paginate($perPage);

// Order detail (admin or tracking)
Order::with(['items.product:id,name,slug', 'shippingMethod:id,name,price', 'statusHistories'])
     ->findOrFail($id);

// Review listing
Review::with('images')->where('product_id', $id)->where('is_approved', true)->paginate(10);
```

Failure to include these eager loads on any endpoint handling collections will produce N+1 queries per row, which degrades linearly with catalog size.

Technical Execution Plan — Bloom Parfums
Version 1.0 | February 25, 2026
Architecture finalized. No revisions pending. Ready for implementation.